BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Quiz 2 Duration: 30 min
Semester: Spring 2025 Full Marks: 15

CSE 470: Software Engineering

Name: **ID:** **Section:**

A legacy banking system needs to modernize its transaction processing while maintaining a **single global transaction ledger**. The **central** ledger must remain **consistent** across all branches, but it needs to handle transactions from **both modern REST-based payment systems and older SOAP-based banking terminals**. Different payment systems use **varying transaction formats,** but they must all be processed and recorded **uniformly in the centralized ledger.**

Python

```
class Ledger:
    def process(self, payment, amount):
        transaction = payment.make_payment(amount)
        print(f"Processed: {transaction}")

class RestPayment:
    def make_payment(self, amount):
        return f"REST Payment: ${amount:.2f}"

class SoapPayment:
    def execute_soap_transfer(self, value):
        return f"SOAP Transfer Executed: ${value:.2f}"

if __name__ == "__main__":
    # Get central ledger instance
    ledger = Ledger()

    # Process modern REST transaction
    rest = RestPayment()
    ledger.process(rest, 500.00)  # This works fine

    # Old SOAP system needs adaptation
    soap = SoapPayment()
    ledger.process(soap, 750.00)  # This will fail
```

Java

```
public class Ledger {
    public void process(Payment payment, double amount){
        String transaction = payment.makePayment(amount);
        System.out.println("Processed: " + transaction);
    }
}

class RestPayment implements Payment {
    public String makePayment(double amount) {
        return "REST Payment: $" + amount;
    }
}
class SoapPayment {
    public String executeSoapTransfer(double value) {
        return "SOAP Transfer Executed: $" + value;
    }
}

interface Payment {
    String makePayment(double amount);
}
public class Main {
    public static void main(String[] args) {
        // Get central ledger instance
        Ledger ledger = new Ledger();

        // Process modern REST transaction
        RestPayment rest = new RestPayment();
        ledger.process(rest, 500);  // This works fine

        SoapPayment soap = new SoapPayment();
        ledger.process(soap, 750);  // This will fail
    }
}
```

1.	**CO3**	a. Considering the above scenario, choose appropriate design pattern(s). Justify your answer. [4 Marks] b. Write the code necessary to complete the provided driver code by implementing your chosen design pattern(s). (Write only the classes you will add, and modify existing classes, **no need to write the main method**. You must implement the method names as given in the scenario.) [8 Marks] c. What are the different categories of Design Patterns? Name a design pattern in each category. [3 Marks]	**15**

a. The appropriate design patterns are Singleton and Adapter pattern.
   Singleton is needed because
   - Single global transaction ledger **(one single instance, shared globally)**
   - Must be processed uniformly in the centralized ledger **(central, consistency)**

   Adapter is needed because
   - Needs to handle transactions from both modern and legacy systems **(has two interfaces, needs to work with both, and one can't be modified)**
   - They have different transaction formats **(the formats are incompatible)**

b. Code (object adapter is better here, since we can expect to have the soap object, but class adapter is fully acceptable)

```
class Ledger:
    _instance = None
    def __new__(cls)
        if _instance is None:
            cls._instance = super(Ledger, cls).__new__(cls)
        return cls._instance

class SoapAdapter():
    def __init__(self, soap_payment_object):
        self.soap = soap_payment_object

    def make_payment(self, amount):
        self.soap.execute_soap_transfer(amount)

# Old SOAP system needs adaptation
    soap = SoapPayment()
    adapter = SoapAdapter(soap)
    ledger.process(adapter, 750.00)  # This will succeed
```

Class Adapter

```
class Ledger:
    _instance = None
    def __new__(cls)
        if _instance is None:
            cls._instance = super(Ledger, cls).__new__(cls)
        return cls._instance

class SoapAdapter(SoapPayment):
    def make_payment(self, amount):
        self.execute_soap_transfer(amount)

# Old SOAP system needs adaptation
    adapter = SoapAdapter()
    ledger.process(adapter, 750.00)  # This will succeed
```

c. The following are the three categories and examples
    i. Creational - Singleton
    ii. Behavioral - Observer
    iii. Structural - Adapter